SECTION 02 · REVIEWS AND LEADS INTO COLUMNS

Product Review Dashboard

Turn free-text reviews and sales notes into typed columns you can chart, filter and rank.

Schema design | Speculative fan-out | Async batch | Dashboard | Lead scoring

KGP Talkie

Scan to subscribe

Laxmi Kant Tiwari · kgptalkie.com

From Review Text to Columns

Review text 120 rows

- Updated to firmware 4.2 and now the battery is dead by evening??
- Sound is honestly insane for the price. Bass is punchy...
- Bhai full value for money. Phone 4-5 baar charge ho jata hai.

unstructured

Your questions the schema

Choice	aspect
Score	sentiment
Noul	competitor
Noul	refund
Noul	recommend

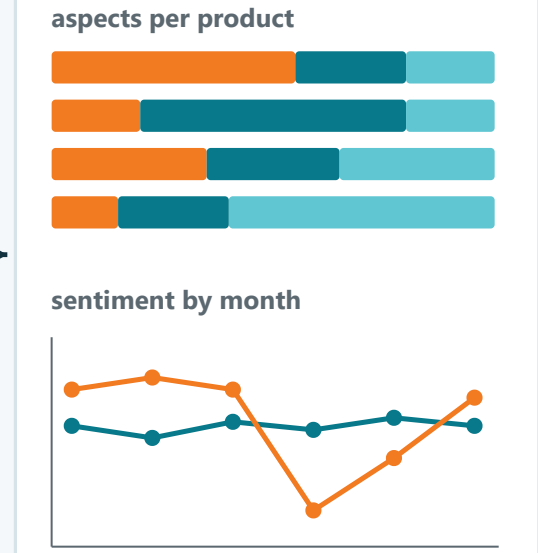
you write these once

Typed columns DataFrame

aspect	category
aspect_conf	0 to 1
sentiment	0 to 4
refund	P(yes)
competitor	P(yes)
recommend	P(yes)

one row per review

Dashboard charts



pandas + matplotlib

Write the questions once; every row of text comes back as the same **typed columns**.

Who Needs Text Turned Into Columns

Product analyst



"What are people complaining about, per product?"

`aspect` crosstab

reviews dashboard

Product manager



"Did last month's firmware update hurt us?"

`sentiment` by month

reviews dashboard

Support lead



"Who is asking for money back right now?"

`refund` > 0.5

refund queue

Sales ops



"Which inbound leads do reps call first?"

`lead_score` sort

lead scoring lab

Each person needs a **column**, not a summary: something to count, average, filter or sort.

Each Question Becomes a Column

QUESTION	ANSWER FIELD	COLUMN TYPE	WHAT PANDAS DOES WITH IT
Choice aspect	.choice	category one of your labels	value_counts() crosstab(product, aspect)
same Choice	.confidence	number 0 to 1 how peaked the choice is	filter rows for a person
Score sentiment	.score	ordered number 0 to n-1, can sit between levels	mean() pivot_table by month
Noul refund	.noul	probability P(yes), 0 to 1	mean() = expected share > 0.5 for a yes/no column

Pick the question type by what you will do with the column in pandas.

Ask One Thing per Question

One vague question

"What does this customer think of the product?"

A paragraph of opinion.
No fixed options, no number,
nothing to group or average.
You would have to parse it again.

not a column

One snap judgment per question

Choice	Which aspect is this review mainly about?	battery, sound, ... overall
Score	How does the reviewer feel overall?	5 described levels
Noul	Mentions a competing brand?	true / false defined
Noul	Asks for a return, refund or replacement?	
Noul	Would recommend it to a friend?	

each answer lands in its own column

A question a person could answer in a second is a question that fills a clean column.

The Aspect Column and Its Confidence

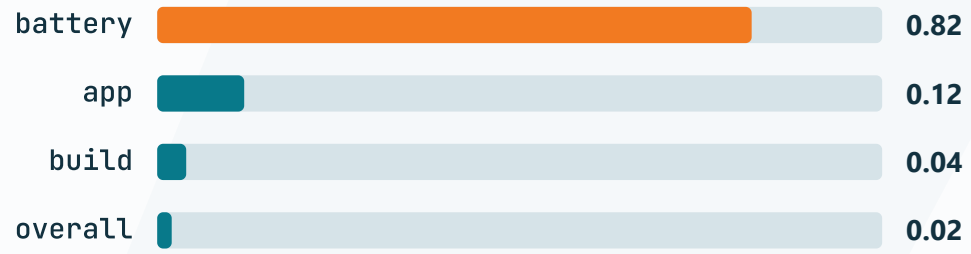
criteria = {label: description}

battery	battery life, charging, drain
sound	audio, bass, volume, ANC, mic
comfort	fit, strap, ear tips
build	durability, defects, breaking
price	price, value for money
app	app, pairing, sync, firmware
delivery	shipping, courier, packaging
support	service, warranty, returns
overall	no single aspect stands out

overall = the escape option, so a vague review is not forced into a wrong label

Example answer — probabilities stated for this slide

"Updated to firmware 4.2 yesterday and now the battery is dead by evening?? It used to last 5 days."



other 5 labels: 0.00

```
confidence = (p_max - 1/n) / (1 - 1/n)
n = 9 labels, p_max = 0.82
(0.82 - 0.111) / (1 - 0.111) = 0.709 / 0.889
= 0.80
```

→ aspect_conf column

Sentiment on Five Described Levels



answer

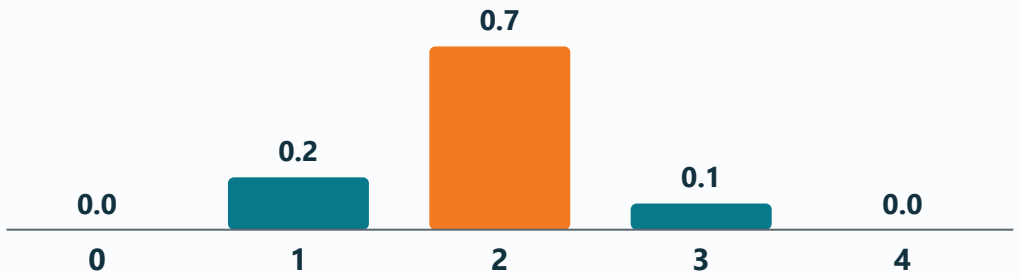
`.probabilities`	one probability per level, they sum to 1
`.score`	a float from 0 to 4, can fall between two levels
`.confidence`	how close the probability mass sits to its peak

A described level is a definition the model can match; a bare word like “mixed” is a guess.

Expected Score vs Most Likely Level

Clear peak

probabilities stated for this slide




```
score = sum(level x p)
      = 1x0.2 + 2x0.7 + 3x0.1 = 1.90

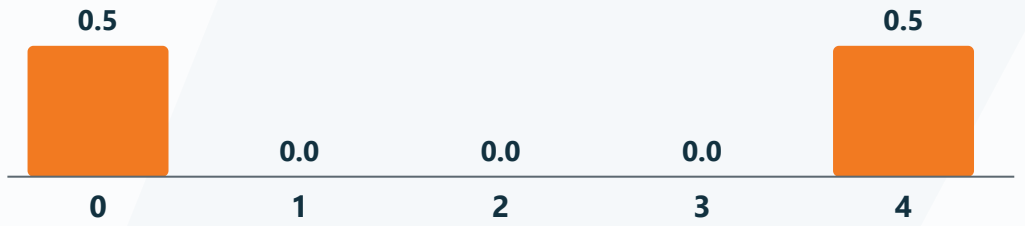
most likely level = 2

spread = sum(p x |level - peak|)
       = 0.30

confidence = max(0, 1 - 0.30 / 1.2) = 0.75
```

Split between the ends

probabilities stated for this slide




```
score = sum(level x p)
      = 0x0.5 + 4x0.5 = 2.00

most likely level = 0 or 4

spread = sum(p x |level - peak|)
       = 2.00

confidence = max(0, 1 - 2.00 / 1.2) = 0.00
```

1.2 = average |level - 2| over five equally likely levels: (2+1+0+1+2)/5

Nouls Give Probability Columns

Noul with criteria

instructions: “The review mentions a competing product from another brand.”

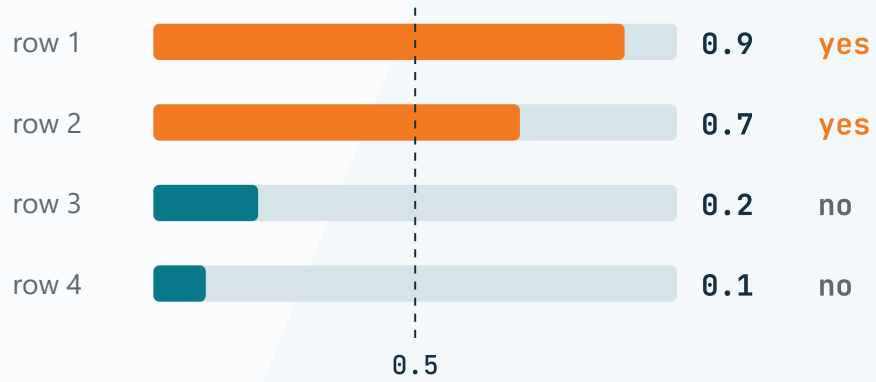
true: another brand's earbuds, watch, power bank, speaker or band

false: no other brand, or only the phone or laptop it is used with

“fast charging works with my Pixel”
is not a competitor: the criteria say so

Two ways to read a probability column

4 example rows



threshold: p > 0.5

2 of 4 rows are yes
share = 0.50
a hard yes/no column,
good for lists and queues

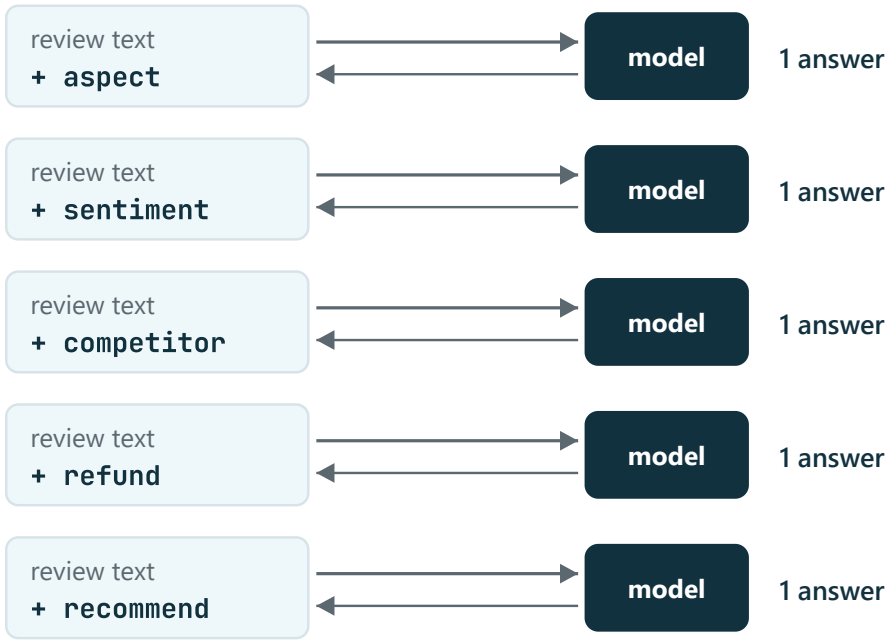
mean: sum(p) / n

(0.9+0.7+0.2+0.1) / 4
expected share = 0.475
keeps the doubt in,
good for KPIs and trends

Average the probabilities for a KPI; threshold them when you need a list of rows.

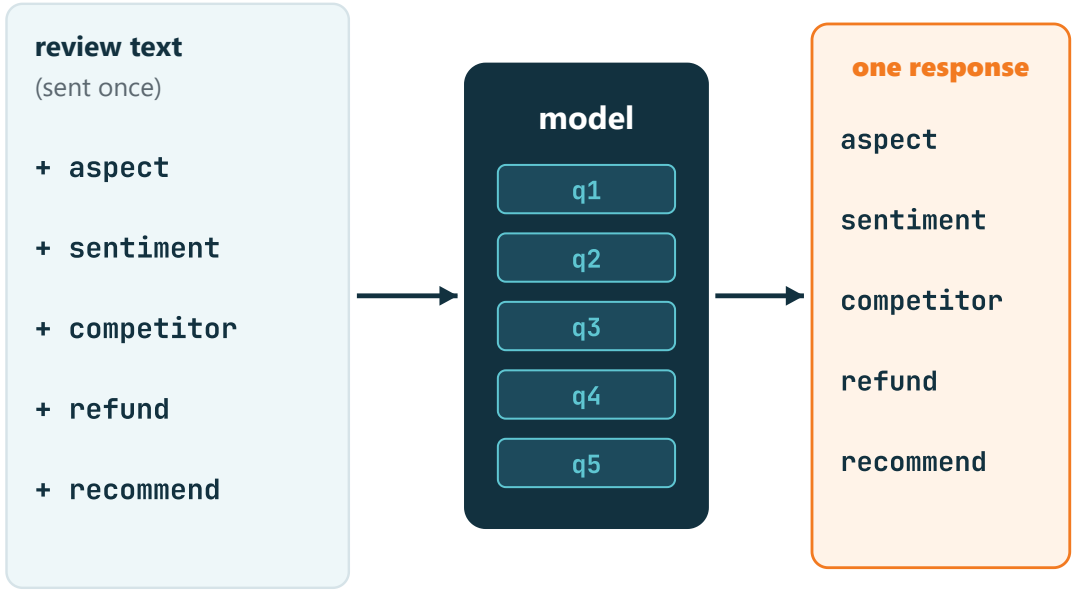
Every Question in One Request

One question per request



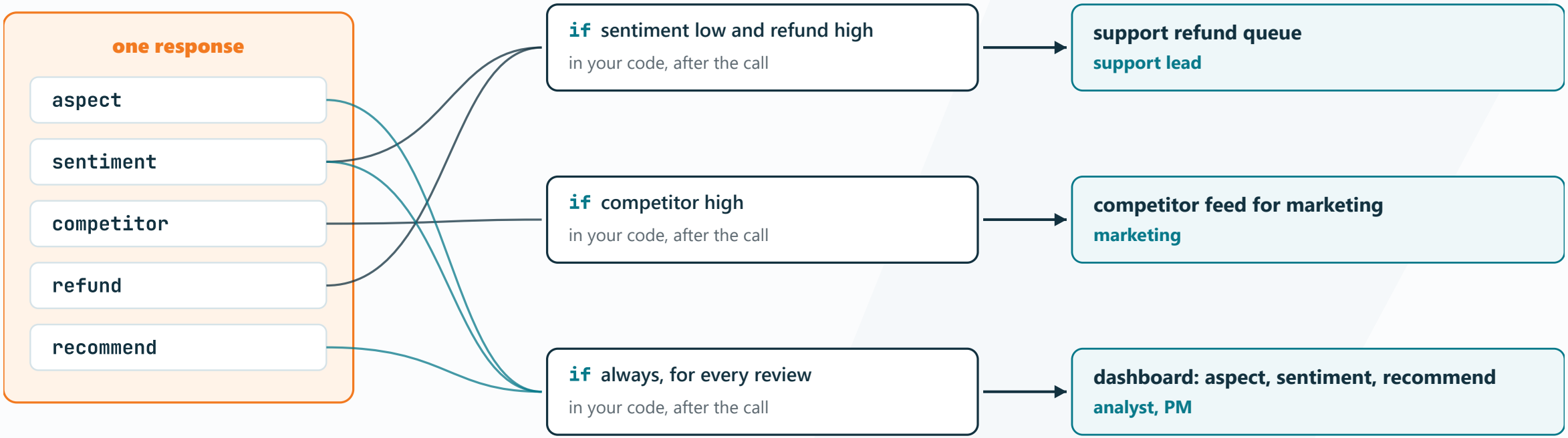
5 round trips, the review is sent 5 times

All questions in one request



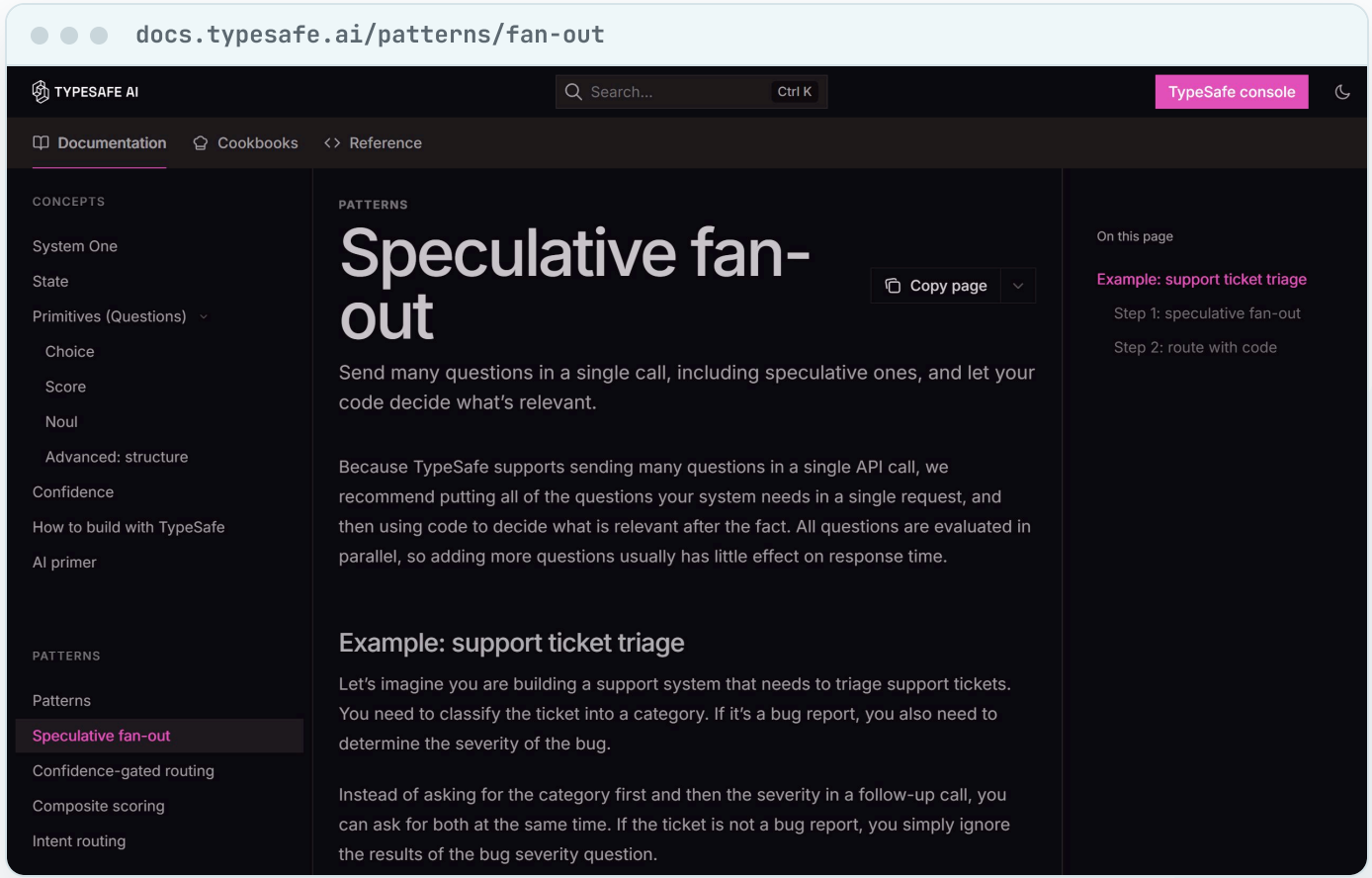
1 round trip, questions answered in parallel

Ask All Questions, Filter in Code



a happy review still carries a refund answer: your code just ignores it

The Fan-Out Pattern in the Docs

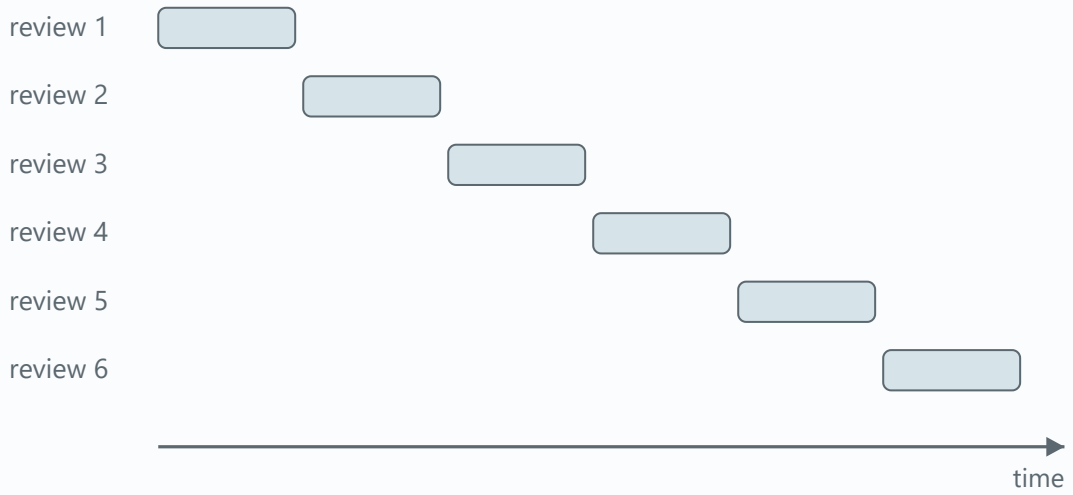


- Put **every** question in one request
- Include questions that matter only for some inputs
- Code ignores the answers it does not need

The docs example is ticket triage; our five review columns follow the same rule.

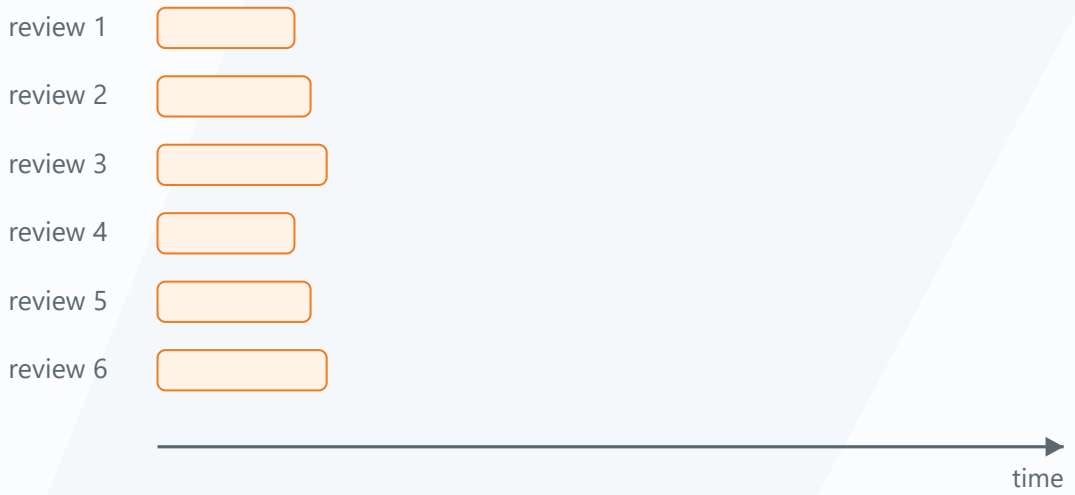
Running All Reviews With asyncio.gather

A loop: one request at a time



total wait = sum of every round trip

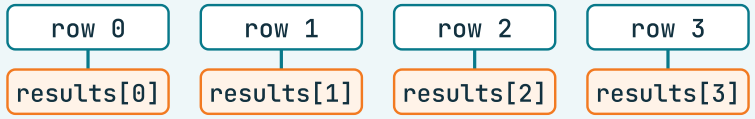
asyncio.gather: all in flight together



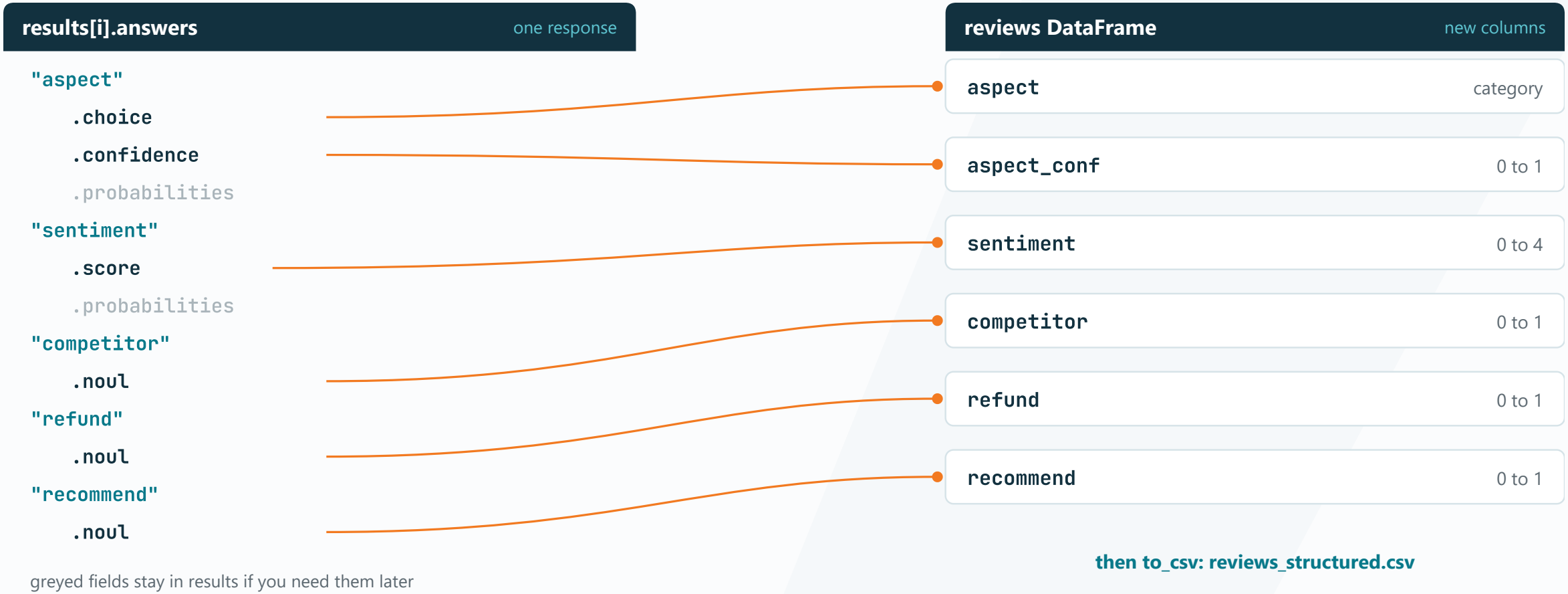
total wait = about the slowest one

gather returns results in input order

results[i] belongs to row i of the DataFrame, so a list comprehension lines them up



From Answers to DataFrame Columns



greyed fields stay in results if you need them later

then to_csv: reviews_structured.csv

Dashboard Charts and What They Show

What are people talking about?

analyst

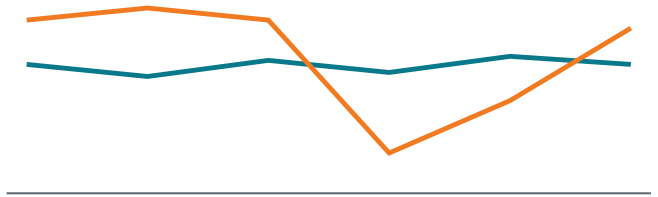


one bar per product,
one colour per aspect

```
crosstab(product, aspect)
```

When did sentiment change?

product manager

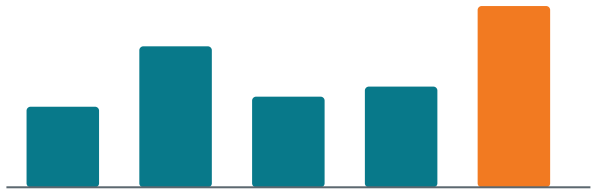


one line per product,
a dip has a month

```
pivot_table(month, product)
```

Where are refunds piling up?

support / ops lead

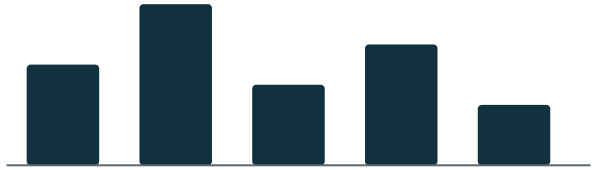


share of reviews that
ask for money back

```
groupby(product): refund > 0.5
```

Who are we compared against?

marketing



a count is a pointer:
read the rows behind it

```
competitor > 0.5, then read them
```

Start from the business question, then pick the one pandas call that answers it.

Average Sentiment by Month

rows — date + sentiment

2026-04-10	Watch S2	0.2
2026-04-19	Watch S2	0.5
2026-04-26	Watch S2	1.1
2026-04-08	Boom Mini	3.9
2026-05-06	Watch S2	0.9

`to_period("M"): 2026-04-10 → 2026-04`

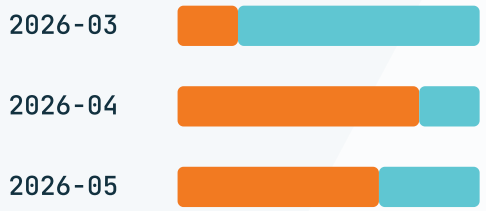
pivot_table — cell = mean

	Boom Mini	Buds Pro	Watch S2
2026-03	...	...	...
2026-04	...	...	0.6
2026-05	...	...	...

`(0.2 + 0.5 + 1.1) / 3 = 0.6`

why? drill down

crosstab(month, aspect) for one product



orange = battery

the monthly mean says WHEN something changed

the aspect column on the same rows says WHAT changed

example values, to show the mechanism

Low Confidence Goes to a Person

Rows sorted by aspect_conf — low first

touch controls pause my music every time I adj...	comfort	
It's ok. Big and heavy. If you need capacity i...	overall	
Exactly what I wanted. Loud, small, lasts all ...	sound	
Charges my phone 4 times. Heavy but it's 20000...	battery	
Sound is honestly insane for the price.	sound	

bar = aspect_conf, dashed line = the 0.5 cut-off

a person checks the label

small queue, read in minutes

straight into the dashboard

no one reads these rows

why a row is unsure (probabilities stated for this slide):

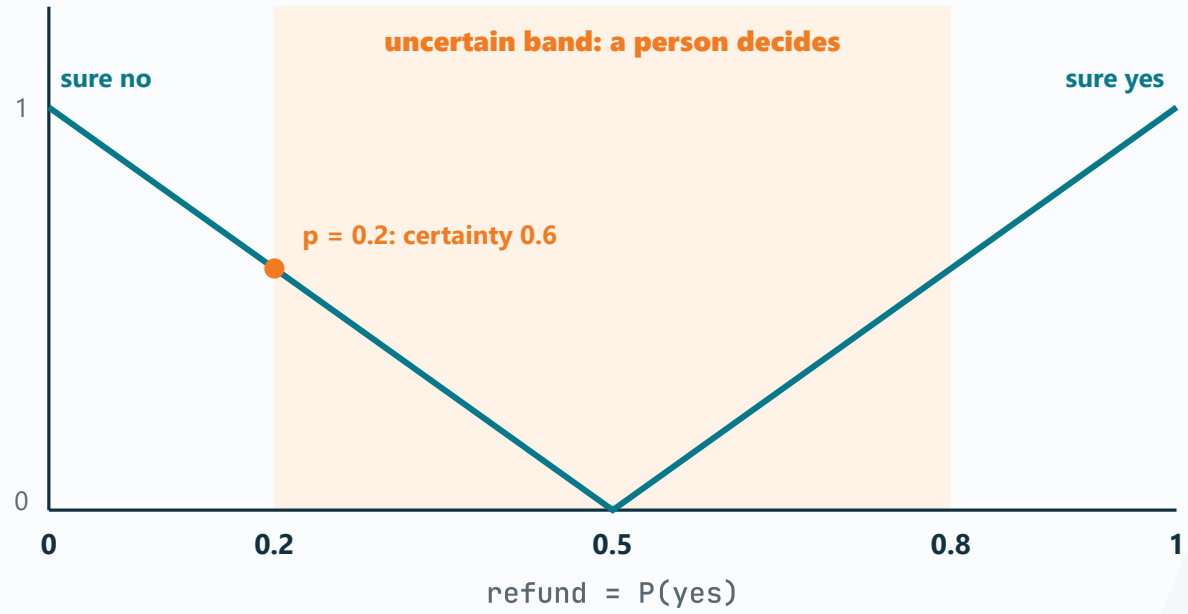
comfort 0.45 app 0.40 build 0.15

$(0.45 - 0.111) / 0.889 = 0.38$

two labels both fit: that is a real ambiguity, not noise

Building the Review Queue

How sure is a Noul? certainty = |2p - 1|



Choice `aspect_conf < 0.5`

the label might be wrong

OR

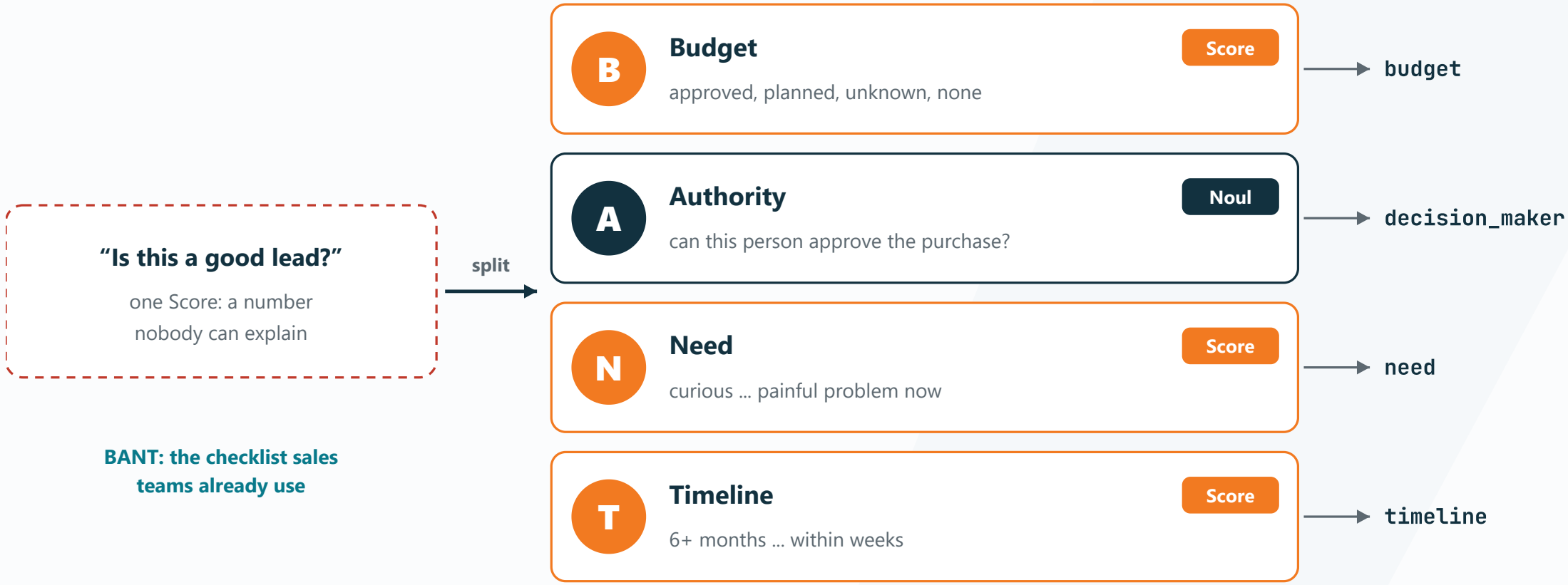
Noul `0.2 ⩽ refund ⩽ 0.8`

might or might not be a refund request

`needs_review = True`

the band and the cut-off are business choices: wider band, bigger queue, fewer silent mistakes

Breaking a Lead Score Into Factors



BANT: the checklist sales teams already use

Four atomic questions, each easy to check against the note, instead of one opaque verdict.

Describe Each Level With Evidence

BUDGET: four levels, and lead notes that read like each one — notes from leads.csv

0	**no budget, or asks for free / cheapest**	“Is there a free plan? Can't really spend much right now.”
1	**budget unknown or not yet allocated**	“Not sure what budget we have, I'd need to ask my lead.”
2	**planned, needs approval or a later fiscal year**	“Budget is in next fiscal year's plan, starting April.”
3	**budget approved and stated**	“Has an approved budget of around $60k a year.”

`state = {"role": ..., "note": ...}` — **Authority reads `role` too**

instructions point at a field with backticks: “How strong is the budget signal in `note`?”

Calculating a Weighted Lead Score



the model answers the four small questions

the weights are yours: a plain line of Python you can read, argue about and change

weights 0.30 + 0.25 + 0.25 + 0.20 = 1, so the score stays between 0 and 1; example answers stated for this slide

Changing Weights to Re-Rank Leads

Standard weights

0.30 / 0.25 / 0.25 / 0.20

Lead A
budget approved, buys next half



`0.300 + 0.250 + 0.083 + 0.200 = 0.83`

Lead B
small budget, needs it this month



`0.100 + 0.167 + 0.250 + 0.200 = 0.72`

Lead A ranks first

Quarter-end push

0.15 / 0.15 / 0.50 / 0.20

Lead A
budget approved, buys next half



`0.150 + 0.150 + 0.167 + 0.200 = 0.67`

Lead B
small budget, needs it this month



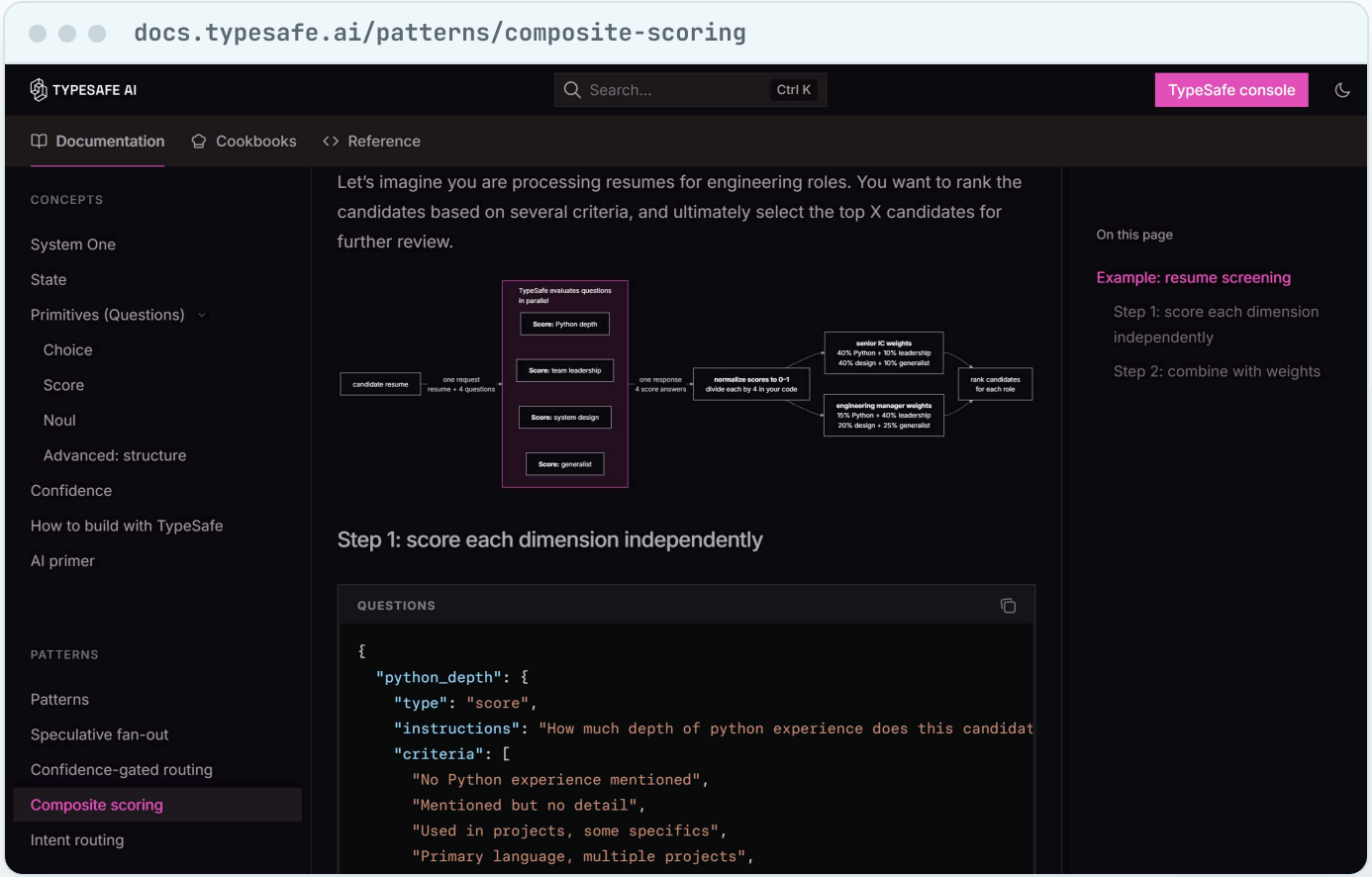
`0.050 + 0.100 + 0.500 + 0.200 = 0.85`

Lead B ranks first

budget · need · timeline · decision_maker

same four answers per lead, no new API calls: only one line of Python changed

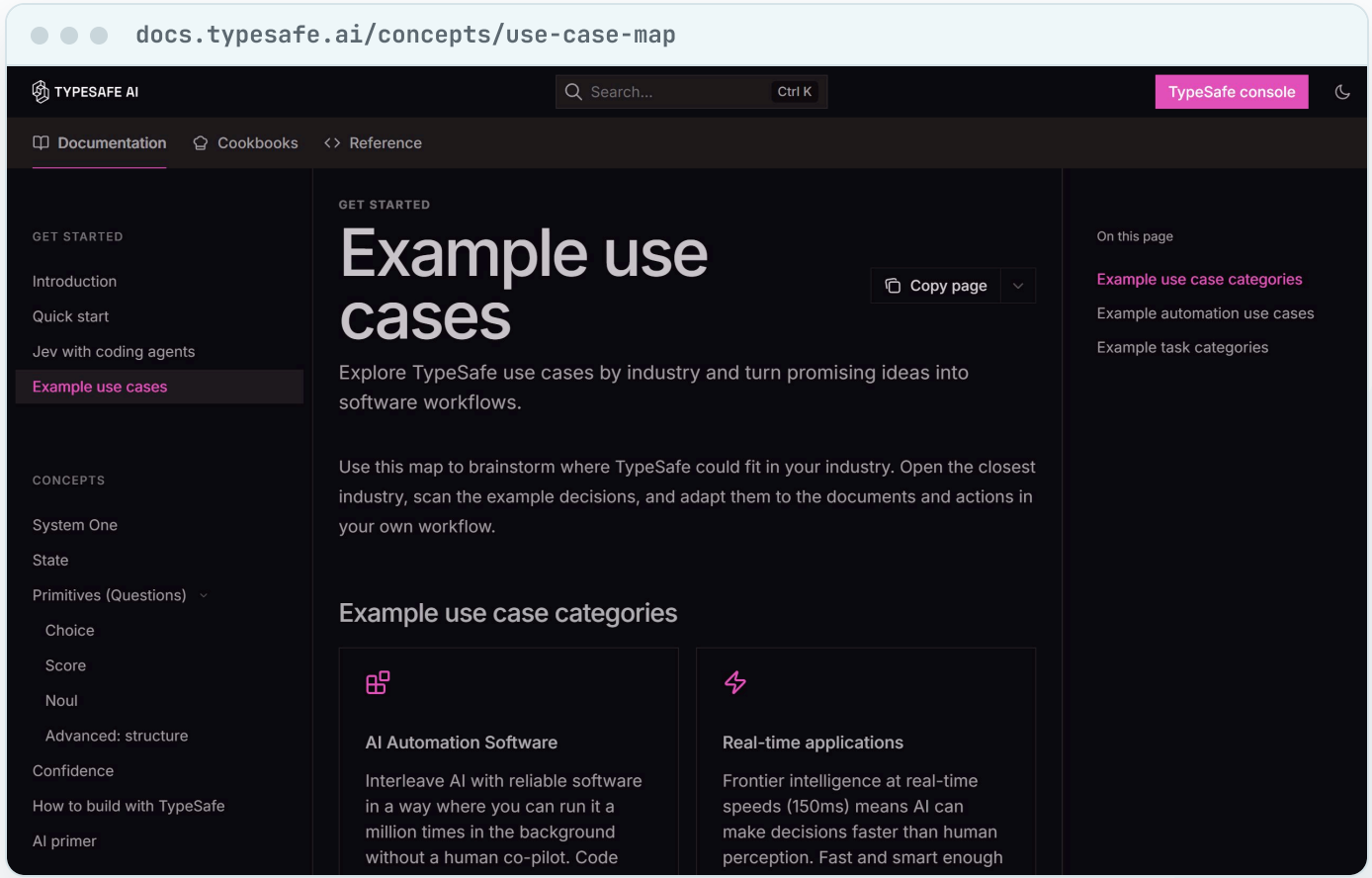
Composite Scoring in the Docs



- Score each dimension **separately**
- Normalise to 0 to 1 in code
- One weight set per goal, then rank

The docs rank resumes for two roles from the same four answers.

Other Industries That Use This



- **Lead generation:** fit, pain points, intent
- **Support:** issue, urgency, refund requests
- **E-commerce:** attributes from listings

Any pile of text you want to **count, chart or rank** is a candidate.

SECTION 02 · COMPLETE

Thank You

Follow KGP Talkie for new lessons, course updates and live sessions.

Scan to follow

Instagram
@kgptalkie

WhatsApp Channel
KGP Talkie